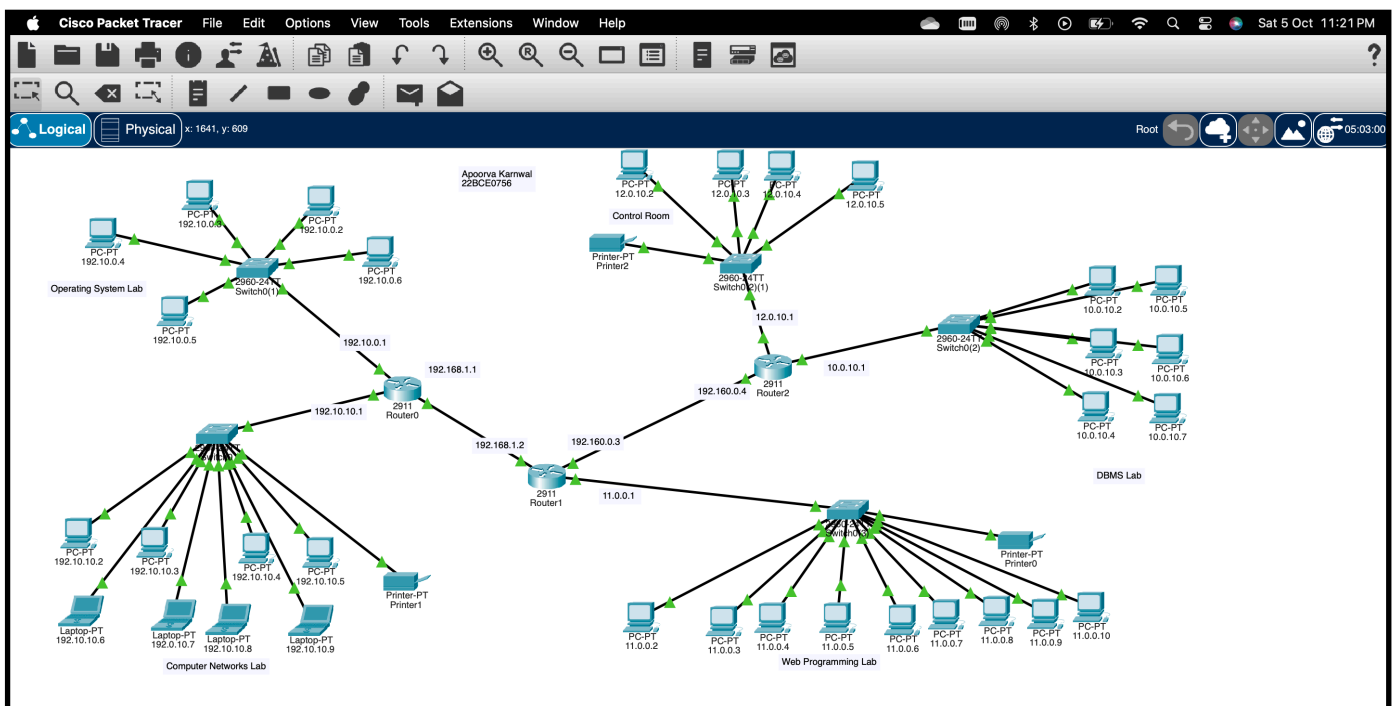
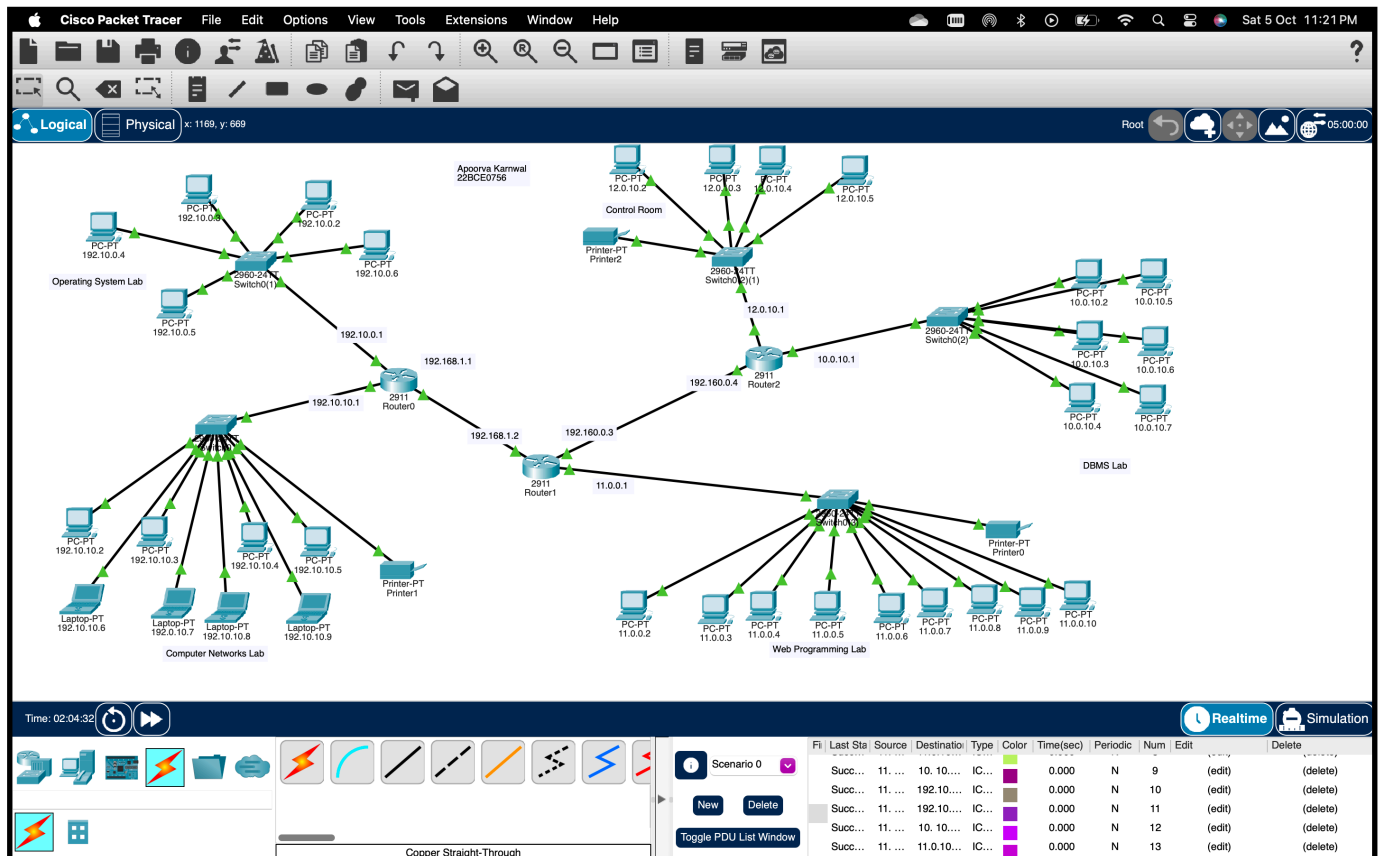
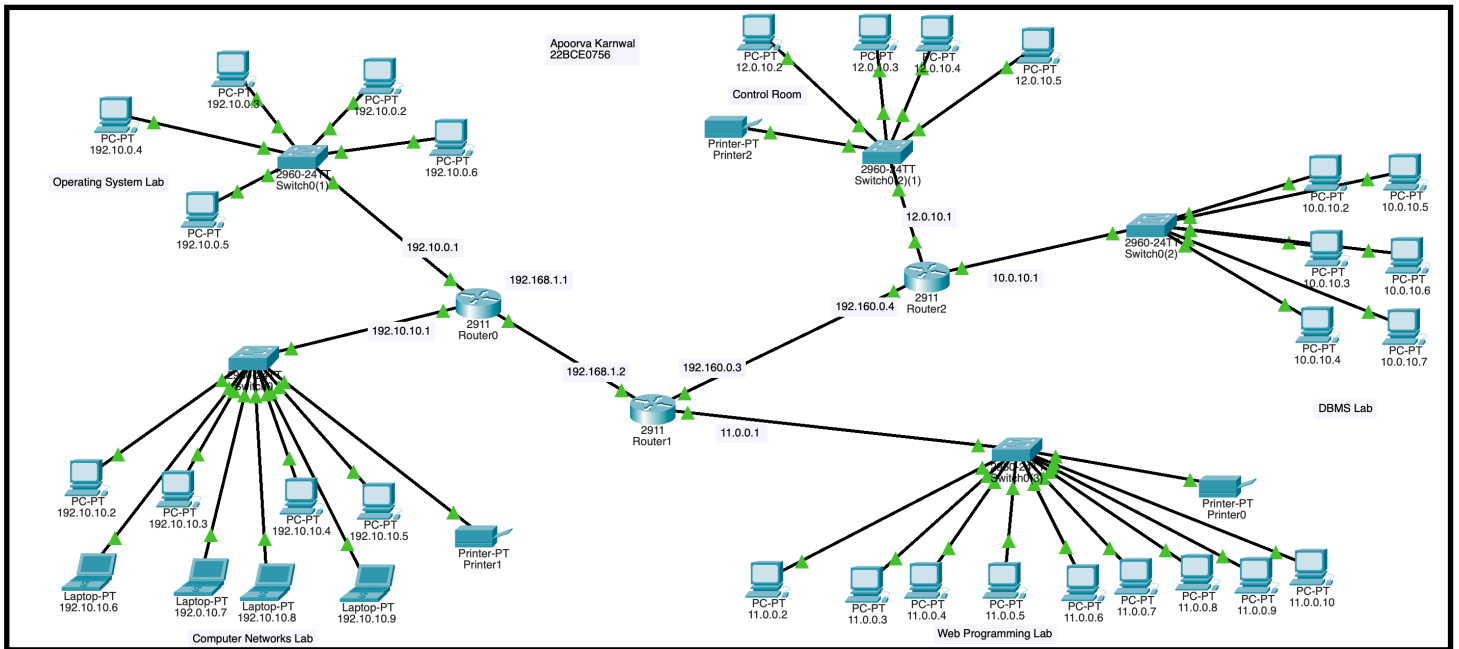


COMPUTER NETWORKS LAB ASSESSMENT - 4

SUBMITTED BY:
APOORVA KARNWAL (22BCE0756)

Q2. Design the campus network using Packet Tracer.





OUTPUT:

Logical

Physical

x: 698, y: 482

The diagram illustrates a complex network topology across several labs:

- Operating System Lab:** Contains PC-PTs with IP addresses 192.10.0.4, 192.10.0.5, 192.10.0.6, and 192.10.0.2, connected to a 2960-24TT Switch0(1).
- Control Room:** Contains PC-PTs with IP addresses 12.0.10.2, 12.0.10.3, 12.0.10.4, and 12.0.10.5, connected to a 2960-24TT Switch0(2)(1).
- DBMS Lab:** Contains PC-PTs with IP addresses 10.0.10.2 through 10.0.10.7, connected to a 2960-24TT Switch0(2).
- Web Programming Lab:** Contains PC-PTs with IP addresses 11.0.0.2 through 11.0.0.10, and a Printer-PT (Printer0), connected to a central 2960-24TT Switch0(2).
- Core Network:** Features three 2911 Routers (Router0, Router1, Router2) and a 2960-24TT Switch0(1) connected to a 2960-24TT Switch0(2). The routers are configured with IP addresses 192.168.1.1, 192.168.1.2, and 192.160.0.3 respectively.

PDU List Window

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	192.10...	192.10.0.6	ICMP		0.000	N	0	(edit)	(delete)
	Successful	11. 0.0.3	192.10.10.4	ICMP		0.000	N	1	(edit)	(delete)
	Successful	10.0.10.2	12.0.10.5	ICMP		0.000	N	2	(edit)	(delete)
	Successful	12.0.10.5	10.0.10.2	ICMP		0.000	N	3	(edit)	(delete)
	Successful	192.10.0.2	192.10.10.5	ICMP		0.000	N	4	(edit)	(delete)
	Successful	10.0.10.3	12.0.10.5	ICMP		0.000	N	5	(edit)	(delete)
	Successful	192.10....	192.10.10.5	ICMP		0.000	N	6	(edit)	(delete)
	Successful	11. 0.0.3	11.0.10. 4	ICMP		0.000	N	7	(edit)	(delete)
	Successful	11. 0.0.3	11.0.10. 4	ICMP		0.000	N	8	(edit)	(delete)
	Successful	11. 0.0.3	10. 10.10.3	ICMP		0.000	N	9	(edit)	(delete)
	Successful	11. 0.0.3	192.10.10.5	ICMP		0.000	N	10	(edit)	(delete)
	Successful	11. 0.0.3	192.10.10.9	ICMP		0.000	N	11	(edit)	(delete)
	Successful	11. 0.0.3	10. 10.10.3	ICMP		0.000	N	12	(edit)	(delete)
	Successful	11. 0.0.3	11.0.10. 4	ICMP		0.000	N	13	(edit)	(delete)

Time: 02:11:26

QUESTION 3: Implement a program to allocate the IP addresses to the Enterprise for the sample Scenario

CODE:

```
#include <stdio.h>
#define TOTAL_ADDRESSES 65536
typedef struct {
    char start_address[16];
    char end_address[16];
    int customers;
    int addresses_per_customer;
} Group;

void allocate_ip_addresses(Group *group, int *base_address) {
    int total_addresses = group->customers * group-
>addresses_per_customer;
    sprintf(group->start_address, "190.100.%d.0", *base_address /
256);
    int last_octet = (*base_address + total_addresses) - 1;
    sprintf(group->end_address, "190.100.%d.%d", last_octet / 256,
last_octet % 256);
    *base_address += total_addresses; // Update base address for
the next group
}

int main() {
    Group groups[] = {
        {"", "", 64, 256},
        {"", "", 128, 128},
        {"", "", 128, 64}
    };
    int base_address = 0; // Starting at 190.100.0.0
    for (int i = 0; i < 3; i++) {
        allocate_ip_addresses(&groups[i], &base_address);
    }
    for (int i = 0; i < 3; i++) {
        printf("Group %d:\n", i + 1);
        printf("Starting Address: %s\n", groups[i].start_address);
        printf("Ending Address: %s\n\n", groups[i].end_address);
    }
    int total_allocated = (groups[0].customers *
groups[0].addresses_per_customer) +
        (groups[1].customers *
groups[1].addresses_per_customer) +
        (groups[2].customers *
groups[2].addresses_per_customer);
    int remaining_addresses = TOTAL_ADDRESSES - total_allocated;
    printf("Remaining Addresses: %d\n", remaining_addresses);
    return 0;
}
```

CODE SCREENSHOT

```
C ip.c > ...
1  #include <stdio.h>
2  #define TOTAL_ADDRESSES 65536
3  typedef struct {
4      char start_address[16];
5      char end_address[16];
6      int customers;
7      int addresses_per_customer;
8  } Group;
9
10 void allocate_ip_addresses(Group *group, int *base_address) {
11     int total_addresses = group->customers * group->addresses_per_customer;
12     sprintf(group->start_address, "190.100.%d.0", *base_address / 256);
13     int last_octet = (*base_address + total_addresses) - 1;
14     sprintf(group->end_address, "190.100.%d.%d", last_octet / 256, last_octet % 256);
15     *base_address += total_addresses;
16 }
17
18 int main() {
19     Group groups[] = { ...
24     int base_address = 0; // Starting at 190.100.0.0
25     for (int i = 0; i < 3; i++) {
26         allocate_ip_addresses(&groups[i], &base_address);
27     }
28     for (int i = 0; i < 3; i++) {
29         printf("Group %d:\n", i + 1);
30         printf("Starting Address: %s\n", groups[i].start_address);
31         printf("Ending Address: %s\n\n", groups[i].end_address);
32     }
33     int total_allocated = (groups[0].customers * groups[0].addresses_per_customer) +
34                          (groups[1].customers * groups[1].addresses_per_customer) +
35                          (groups[2].customers * groups[2].addresses_per_customer);
36     int remaining_addresses = TOTAL_ADDRESSES - total_allocated;
37     printf("Remaining Addresses: %d\n", remaining_addresses);
38     return 0;
39 }
40
```

OUTPUT

```
> ▾ TERMINAL
ip && "/Users/karnwalpoorva23/Desktop/hack/"ip
Group 1:
Starting Address: 190.100.0.0
Ending Address: 190.100.63.255

Group 2:
Starting Address: 190.100.64.0
Ending Address: 190.100.127.255

Group 3:
Starting Address: 190.100.128.0
Ending Address: 190.100.159.255

Remaining Addresses: 24576
```

QUESTION4 Implement Link State Routing Algorithm/Distance Vector Routing for the topology given and construct routing a=table for any node.

```
#include <stdio.h>
#define V 9
#define INF 9999
struct Node {
    int dist[V];
    int from[V];
};

void distanceVectorRouting(int costMatrix[V][V]) {
    struct Node routingTable[V];
    int i, j, k;
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            routingTable[i].dist[j] = costMatrix[i][j];
            if (costMatrix[i][j] != INF && i != j) {
                routingTable[i].from[j] = j;
            } else {
                routingTable[i].from[j] = -1;
            }
        }
    }

    int updated;
    do {
        updated = 0;
        for (i = 0; i < V; i++) {
            for (j = 0; j < V; j++) {
                for (k = 0; k < V; k++) {
                    if (routingTable[i].dist[j] >
routingTable[i].dist[k] + routingTable[k].dist[j]) {
                        routingTable[i].dist[j] =
routingTable[i].dist[k] + routingTable[k].dist[j];
                        routingTable[i].from[j] = k;
                        updated = 1;
                    }
                }
            }
        }
    } while (updated);
    for (i = 0; i < V; i++) {
        printf("Routing table for node %c:\n", 'A' + i);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < V; j++) {
            if (routingTable[i].dist[j] == INF) {
                printf("%c\t\t-\t\tINF\n", 'A' + j);
            } else {
                printf("%c\t\t%c\t\t%d\n", 'A' + j,
routingTable[i].from[j] == -1 ? '-' : 'A' +
routingTable[i].from[j], routingTable[i].dist[j]);
            }
        }
    }
}
```

```

        printf("\n");
    }
}
int main() {
    int costMatrix[V][V] = {
        { 0, 4, 8, INF, INF, INF, INF, INF, INF },//a
        { 4, 0, 11, 8, INF, INF, INF, INF, INF },//b
        { 8, 11, 0, INF, INF, 1, INF, INF, INF },//c
        { INF, 8, INF, 0, 2, INF, 7, 4, INF },//d
        { INF, INF, 7, 2, 0, 6, INF, INF, INF },//e
        { INF, INF, 1, INF, 6, 0, INF, 2, INF },//f
        { INF, INF, INF, 7, INF, INF, 0, 14, 9 },//g
        { INF, INF, INF, 4, INF, 2, 14, 0, 10 },//h
        { INF, INF, INF, INF, INF, INF, 9, 10, 0 }//i
    };
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (costMatrix[i][j] == 0 && i != j) {
                costMatrix[i][j] = INF;
            }
        }
    }
    distanceVectorRouting(costMatrix);
    return 0;
}

```

C dvr.c > distanceVectorRouting(int [V][V])

```

1  #include <stdio.h>
2  #define V 9
3  #define INF 9999
4  struct Node {
5      int dist[V];
6      int from[V];
7  };
8  void distanceVectorRouting(int costMatrix[V][V]) {
9      struct Node routingTable[V];
10     int i, j, k;
11     for (i = 0; i < V; i++) {
12         for (j = 0; j < V; j++) {
13             routingTable[i].dist[j] = costMatrix[i][j];
14             if (costMatrix[i][j] != INF && i != j) {
15                 routingTable[i].from[j] = j;
16             } else {
17                 routingTable[i].from[j] = -1;
18             }
19         }
20     }
21     int updated;
22     do {
23         updated = 0;
24         for (i = 0; i < V; i++) {
25             for (j = 0; j < V; j++) {
26                 for (k = 0; k < V; k++) {
27                     if (routingTable[i].dist[j] > routingTable[i].dist[k] + routingTable[k].dist[j]) {
28                         routingTable[i].dist[j] = routingTable[i].dist[k] + routingTable[k].dist[j];
29                         routingTable[i].from[j] = k;
30                         updated = 1;
31                     }
32                 }
33             }
34         }
35     } while (updated);
36     for (i = 0; i < V; i++) {
37         printf("Routing table for node %c:\n", 'A' + i);
38         printf("Destination\tNext Hop\tDistance\n");
39         for (j = 0; j < V; j++) {
40             printf("%d\t\t", routingTable[i].from[j]);
41             if (j % 10 == 9) printf("\n");
42         }
43     }
44 }

```

```

39     for (j = 0; j < V; j++) {
40         if (routingTable[i].dist[j] == INF) {
41             printf("%c\t\t\t\t\tINF\n", 'A' + j);
42         } else {
43             printf("%c\t\t\t\t\t%d\n", 'A' + j, routingTable[i].from[j] == -1 ? '-' : 'A' + routingTable[i].from[j],
44                 routingTable[i].dist[j]);
45         }
46     }
47     printf("\n");
48 }
49 }
50 int main() {
51     int costMatrix[V][V] = {
52         { 0, 4, 8, INF, INF, INF, INF, INF, INF }, //a
53         { 4, 0, 11, 8, INF, INF, INF, INF, INF }, //b
54         { 8, 11, 0, INF, INF, 1, INF, INF, INF }, //c
55         { INF, 8, INF, 0, 2, INF, 7, 4, INF }, //d
56         { INF, INF, 7, 2, 0, 6, INF, INF, INF }, //e
57         { INF, INF, 1, INF, 6, 0, INF, 2, INF }, //f
58         { INF, INF, INF, 7, INF, INF, 0, 14, 9 }, //g
59         { INF, INF, INF, 4, INF, 2, 14, 0, 10 }, //h
60         { INF, INF, INF, INF, INF, INF, 9, 10, 0 } //i
61     };
62     for (int i = 0; i < V; i++) {
63         for (int j = 0; j < V; j++) {
64             if (costMatrix[i][j] == 0 && i != j) {
65                 costMatrix[i][j] = INF;
66             }
67         }
68     }
69     distanceVectorRouting(costMatrix);
70     return 0;
71 }

```

OUTPUT

```

>  TERMINAL
Routing table for node A:
Destination  Next Hop  Distance
A            -            0
B            B            4
C            C            8
D            B            12
E            D            14
F            C            9
G            D            19
H            F            11
I            H            21

Routing table for node B:
Destination  Next Hop  Distance
A            A            4
B            -            0
C            C            11
D            D            8
E            D            10
F            C            12
G            D            15
H            D            12
I            H            22

Routing table for node C:
Destination  Next Hop  Distance
A            A            8
B            B            11
C            -            0
D            F            7
E            F            7
F            F            1
G            D            14
H            F            3
I            H            13

Routing table for node D:
Destination  Next Hop  Distance
A            B            12
B            B            8
C            F            7
D            -            0
E            E            2
F            H            6
G            G            7
H            H            4
I            H            14

Routing table for node E:
Destination  Next Hop  Distance
A            D            14
B            D            10
C            C            7

```

TERMINAL		
B	D	10
C	C	7
D	D	2
E	-	0
F	F	6
G	D	9
H	D	6
I	D	16
Routing table for node F:		
Destination	Next Hop	Distance
A	C	9
B	C	12
C	C	1
D	H	6
E	E	6
F	-	0
G	D	13
H	H	2
I	H	12
Routing table for node G:		
Destination	Next Hop	Distance
A	D	19
B	D	15
C	D	14
D	D	7
E	D	9
F	D	13
G	-	0
H	D	11
I	I	9
Routing table for node H:		
Destination	Next Hop	Distance
A	F	11
B	D	12
C	F	3
D	D	4
E	D	6
F	F	2
G	D	11
H	-	0
I	I	10
Routing table for node I:		
Destination	Next Hop	Distance
A	H	21
B	H	22
C	H	13
D	H	14
E	D	16
F	H	12
G	G	9
H	H	10
I	-	0